Universidad de La Guajira

Facultad de Ingeniería - Programación Avanzada A1

Proyecto Final: Aplicación Real con Python

Docente:	Jenier Javier Suarez Marquez
Período:	20262
Fecha límite:	Miércoles 25 de noviembre de 2026, 12:00 m. (mediodía)
Modalidad:	Grupos de hasta **4 personas** (mín. 2)
Entrega:	Repositorio Git + Sustentación presencial

Descripción de la Actividad

El proyecto final consiste en construir una **aplicación real en Python** que resuelva un problema concreto. El tema es completamente libre: el grupo define qué quiere construir y para quién. No hay una arquitectura obligatoria ni un patrón de diseño que cumplir — lo que importa es que el producto **funcione, tenga sentido y pueda mostrarse**.

La idea es aprovechar el ecosistema real de Python: frameworks web, consumo de APIs, bases de datos, automatización, inteligencia artificial. Ese es el tipo de habilidad que se usa en la industria y que este proyecto busca practicar.

> **No se aceptan aplicaciones de solo consola** (`input`/`print`). El proyecto debe tener una interfaz o punto de acceso real: web, API, bot o escritorio.

¿Qué tipo de proyecto se espera?

Se recomienda fuertemente construir sobre tecnologías web y de backend. Algunos caminos naturales, de menor a mayor complejidad:

- **API REST con FastAPI o Django REST Framework** — backend con endpoints, base de datos (SQLite o PostgreSQL), documentación automática con Swagger. El frontend puede ser simple HTML o incluso Postman como cliente.
- **Aplicación web completa con Django** — vistas, modelos, autenticación de usuarios, base de datos, templates. Un proyecto Django bien hecho ya es un portafolio real.
- **Integración con APIs externas** — consumir servicios reales (clima, mapas, pagos, WhatsApp Business, Spotify, YouTube Data, etc.) y construir algo útil encima de ellos.
- **Bot funcional** — Telegram o Discord con comandos reales, persistencia de datos y lógica de negocio no trivial.

- **App de escritorio con interfaz** — `CustomTkinter`, `PyQt6` o `Flet` para herramientas de productividad, dashboards locales, etc.
- **Pipeline de datos o IA** — scraping + procesamiento + visualización web (`Streamlit`, `Gradio`, `Dash`), o integración de un modelo de lenguaje o visión en una app con interfaz.

Ideas de proyectos (solo como inspiración)

- API de gestión de turnos o reservas para un negocio local (restaurante, clínica)
- App Django para control de préstamos de libros o equipos de laboratorio
- Dashboard web con datos de una API pública (clima, aire, sismos, transporte)
- Bot de Telegram para recordatorios, encuestas o asistente de tareas del curso
- Sistema de inventario con Django + autenticación de usuarios
- Integración con la API de WhatsApp Business para notificaciones automáticas
- App web con FastAPI que consuma un modelo de IA (GPT, Gemini, Whisper, etc.)
- Scraper + API + frontend: datos de empleo, eventos o noticias de La Guajira
- Herramienta web para gestionar presupuestos o gastos personales

¿Se puede usar Inteligencia Artificial?

Sí, sin restricción. Copilot, Claude, ChatGPT, Cursor — todas las herramientas disponibles son válidas. También pueden integrar IA dentro de la app misma. Las condiciones:

- Cada integrante debe poder **explicar cualquier parte del código** durante la sustentación. No entender el código equivale a no haberlo hecho.
- El `README` debe incluir una sección **"Uso de IA"** describiendo qué herramientas se usaron y en qué partes del proyecto.

Entregables

1. **Enlace al repositorio Git** (GitHub, GitLab o Bitbucket) enviado al docente antes del **miércoles 25 de noviembre de 2026 a las 12:00 m. (mediodía)**
2. `README.md` con: nombre del proyecto, descripción del problema que resuelve, instrucciones de instalación y ejecución, uso de IA, y nombres de todos los integrantes.
3. Sustentación presencial el miércoles 25 de noviembre en horario de clase.
   10 minutos por grupo (demo en vivo + preguntas individuales).

   La sustentación es la muestra del proyecto, cada integrante debe demostrar que domina lo que se construyó. Ese es el tiempo máximo, prepara la exposición para que no se pase de este tiempo.

Rúbrica de Calificación

La nota final del proyecto es **5.0 puntos**:

Criterio	Pts	%
1. Utilidad real y alcance del proyecto	1.0	20 %
2. Funcionalidad y ejecución	1.5	30 %
3. Calidad técnica del código	1.0	20 %
4. Sustentación individual	1.0	20 %
5. Repositorio y documentación	0.5	10 %
Total	**5.0**	**100 %**

Criterio 1 — Utilidad Real y Alcance del Proyecto (1.0 pt)

Nivel	Pts	Descripción	Indicadores
Sobresaliente	1.0	El proyecto resuelve un problema real y específico. Alguien que no conoce programación podría usarlo y encontrarle valor.	Hay un usuario claro, el problema está bien delimitado, el alcance es ambicioso pero alcanzado.
Satisfactorio	0.7	El proyecto tiene utilidad reconocible aunque el problema sea genérico o el alcance sea básico para el tamaño del grupo.	La idea es válida pero poco diferenciada o con funcionalidades mínimas para 4 personas.
Básico	0.4	El proyecto es un ejercicio técnico sin propósito claro. Existe pero no resuelve nada concreto para ningún usuario.	“Hicimos una app CRUD” sin contexto de para qué o para quién.
No cumple	0.0	Solo consola, o el proyecto no tiene ninguna funcionalidad verificable.	

Criterio 2 — Funcionalidad y Ejecución (1.5 pts)

Nivel	Pts	Descripción	Indicadores
Sobresaliente	1.5	La app corre sin errores, cubre todos los casos que el grupo prometió y la demo fluye naturalmente.	Datos persisten, hay manejo de errores visible, al menos 3 funcionalidades completas y demostrables.
Satisfactorio	1.0	El flujo principal funciona sin problemas aunque algunas funcionalidades secundarias estén incompletas.	El core de la app no falla en la demo; las partes incompletas son periféricas.
Básico	0.5	La app corre pero la demo requiere esquivar errores o mostrar solo casos muy controlados.	Features prometidas que no están, bugs visibles, flujos sin terminar.
No cumple	0.0	La app no corre o la demo es imposible de realizar.	Errores al iniciar, dependencias faltantes, código que no ejecuta.

Criterio 3 — Calidad Técnica del Código (1.0 pt)

Nivel	Pts	Descripción	Indicadores
Sobresaliente	1.0	Código limpio, organizado por módulos o apps (en Django: apps separadas). Uso apropiado del framework elegido, variables descriptivas, sin código muerto.	Separación clara de lógica y presentación, uso correcto de modelos/vistas/serializadores según el stack, `.env` para variables sensibles.
Satisfactorio	0.7	Código funcional y con estructura razonable aunque con algo de desorden o mezcla de responsabilidades.	Archivos separados con propósito claro, stack usado de forma básica pero correcta.

Nivel	Pts	Descripción	Indicadores
Básico	0.4	Código en un solo archivo o sin estructura de proyecto. Funciona pero es difícil de leer o mantener.	Todo en `app.py`, credenciales hardcodeadas, lógica mezclada con rutas o vistas.
No cumple	0.0	Python se usa de forma marginal o el código no es original en ninguna parte.	

Criterio 4 — Sustentación Individual (1.0 pt)

*Este criterio se evalúa **por persona**. La nota puede diferir entre integrantes del mismo grupo.*

Nivel	Pts	Descripción	Indicadores
Sobresaliente	1.0	Explica con seguridad su parte y el proyecto en general. Responde preguntas técnicas correctamente y puede navegar el código en vivo.	Puede modificar o trazar un flujo en tiempo real, explica por qué se usó tal framework, endpoint o librería.
Satisfactorio	0.7	Explica su parte bien y responde preguntas básicas. Puede dudar en partes del proyecto que desarrolló otro compañero.	Conoce el flujo general, sabe qué hace cada módulo, puede correr la app y navegar el repositorio.
Básico	0.4	Solo describe qué hace la app sin poder entrar en detalles técnicos. Depende de que un compañero responda.	No puede abrir el código, no sabe cómo está estructurado el proyecto, describe features sin saber cómo están implementadas.
No cumple	0.0	No puede explicar nada o es evidente que no participó en el desarrollo.	Respuestas evasivas, no conoce el repositorio, no sabe qué tecnologías se usaron.

Criterio 5 — Repositorio y Documentación (0.5 pt)

Nivel	Pts	Descripción	Indicadores
Completo	0.5	Repositorio público con commits de todos los integrantes, README completo y estructura de proyecto organizada.	`requirements.txt` o `pyproject.toml`, `.gitignore` correcto (sin subir `.env` ni `__pycache__`), mínimo 5 commits por integrante con mensajes descriptivos.
Parcial	0.25	Repositorio existe pero README incompleto, commits de una sola persona o mensajes genéricos.	Un único commit masivo, sin instrucciones de instalación, o se subieron archivos sensibles al repo.
Ausente	0.0	No hay repositorio o la entrega es por otro medio sin control de versiones.	Entrega por Drive, correo o WhatsApp.

Planilla de Calificación

Nombre del proyecto: _____

Criterio	Peso	Nota /5	Ponderado
1. Utilidad real y alcance	20 %	/5	
2. Funcionalidad y ejecución	30 %	/5	
3. Calidad técnica del código	20 %	/5	
4. Sustentación individual*	20 %	/5	
5. Repositorio y documentación	10 %	/5	
TOTAL	100 %		**/5.0**

* El criterio 4 genera notas individuales: cada integrante puede obtener una puntuación distinta según su desempeño en la sustentación.

Política de Entrega

- Fecha límite: Miércoles 25 de noviembre de 2026, 12:00 m. (hora Colombia).
  Entregas recibidas después de las 12:00 m. no serán recibidas ni calificadas.
- **Grupos:** De 2 a 4 integrantes. Grupos de 1 persona deben ser autorizados por el docente hasta el miércoles 4 de noviembre de 2026.
- **Uso de IA:** Permitido completamente. Si en la sustentación se evidencia que un integrante no comprende el código, su nota en el criterio 4 será **0.0** sin importar la calidad técnica del resto del proyecto.
- **Commits:** Todos los integrantes deben tener commits en el historial. Un repositorio con commits de una sola persona puede generar revisión de autoría.
- **Variables sensibles:** Claves de API, contraseñas o tokens **no deben subirse al repositorio**. Usar `.env` con `python-dotenv`. Un repo con credenciales expuestas tendrá descuento en el criterio 5.
- Preguntas: Al correo institucional del docente o al canal del curso, hasta el miércoles 18 de noviembre de 2026.